

Proyecto: Gestión Inteligente de Trafico Urbano

Marianne Coy y Daniel Diaz

Pontificia Universidad Javeriana

I. Introducción

En las ciudades modernas, la gestión eficiente del tráfico urbano es esencial para garantizar la movilidad, reducir la congestión y mejorar la calidad de vida de los ciudadanos. El presente proyecto tiene como objetivo el diseño e implementación de un sistema distribuido para la gestión inteligente del tráfico urbano, utilizando principios fundamentales de sistemas distribuidos, comunicación asíncrona y tolerancia a fallos.

El sistema propuesto tiene como propósito monitorear, analizar y optimizar las condiciones del tráfico en tiempo real, utilizando sensores distribuidos en diferentes intersecciones de la ciudad. La comunicación entre los componentes del sistema se realiza a través de la biblioteca ZeroMQ, lo que asegura una comunicación eficiente y desacoplada entre los diferentes procesos involucrados, como los sensores de tráfico, el servicio de análisis de datos y el control de semáforos.

La arquitectura distribuida propuesta involucra tres componentes principales: PC1, que se encarga de publicar los eventos generados por los sensores; PC2, que procesa los eventos recibidos y consulta el estado del sistema; y PC3, que responde a las solicitudes de monitoreo y gestión de emergencias, como el paso de vehículos prioritarios. El sistema no solo garantiza la optimización del flujo vehicular, sino que también está diseñado para ser resiliente ante fallas, asegurando la continuidad del servicio incluso en escenarios de fallo de componentes clave.

Este informe describe la implementación de los tres componentes del sistema, los patrones de comunicación utilizados y las pruebas realizadas para evaluar el desempeño del sistema bajo diversas condiciones operativas. Además, se incluye el análisis de las métricas de rendimiento y las estrategias para la conmutación en caso de fallas.

II. Objetivos

I. Objetivo General

Diseñar e implementar un sistema distribuido para la gestión inteligente del tráfico urbano, capaz de monitorear, analizar y gestionar las condiciones de tráfico en tiempo real, utilizando comunicación asíncrona, tolerancia a fallos y mecanismos de resiliencia.

II. Objetivos Específicos

Crear una solución arquitectónica distribuida que simule una ciudad con intersecciones controladas por semáforos inteligentes y sensores de tráfico, usando los principios de los sistemas distribuidos.

Utilizar la biblioteca ZeroMQ para la comunicación eficiente entre los componentes del sistema mediante los patrones PUB/SUB y REQ/REP, optimizando la latencia y el rendimiento.

Asegurar que el sistema continúe operando de manera efectiva incluso en caso de fallos en componentes clave, como el servidor de monitoreo (PC3), mediante la conmutación automática hacia bases de datos de respaldo.

Implementar una base de datos distribuida para el almacenamiento y la actualización en tiempo real de los datos de tráfico, asegurando la consistencia y persistencia de la información.

Diseñar un servicio de análisis que pueda tomar decisiones automáticas sobre el control de semáforos en función de las condiciones del tráfico, incluyendo situaciones especiales como el paso de vehículos prioritarios.

Realizar pruebas de rendimiento, latencia y capacidad de procesamiento bajo diversas condiciones de carga, evaluando las métricas clave como el tiempo de respuesta para el cambio de semáforo y el número de eventos procesados por segundo.

III. Modelos del Sistema

A. Modelo Arquitectónico

El sistema distribuido de gestión inteligente del tráfico urbano se compone de tres nodos principales distribuidos, cada uno con responsabilidades específicas dentro de la arquitectura global. Este modelo permite la separación de tareas y facilita la escalabilidad horizontal del sistema, asegurando un procesamiento eficiente y resiliente ante posibles fallas.

- PC1 - Publicador de eventos de tráfico: Este nodo es el encargado de generar y publicar eventos de tráfico en tiempo real. Los sensores distribuidos en diversas intersecciones de la ciudad envían datos periódicos sobre el volumen vehicular, la velocidad promedio y la ocupación de las vías.

Utiliza ZeroMQ para la publicación de eventos mediante el patrón PUB/SUB. Los eventos generados son enviados de manera asíncrona a los otros componentes del sistema, específicamente a PC2. Este nodo contiene los sensores de tráfico (espiras inductivas, cámaras, GPS, semáforos)

- PC2 - Procesador y servicio de consultas: Recibe los eventos de tráfico publicados por PC1, los procesa para detectar condiciones de congestión y toma decisiones sobre el control de los semáforos. Además, realiza consultas periódicas al nodo de monitoreo (PC3) para obtener el estado del sistema y gestionarlo. Utiliza ZeroMQ con el patrón SUB para suscribirse a los eventos de tráfico y el patrón REQ/REP para enviar solicitudes de estado a PC3. Además, este componente actúa como un intermediario entre los sensores y los semáforos, también tiene una réplica de la bases de datos en caso que la otra falle .

PC3 - Monitoreo y gestión de emergencias: Este nodo se encarga del monitoreo general del sistema y responde a las solicitudes enviadas por PC2. Permite la intervención manual en casos de emergencia, como la priorización del paso de vehículos de emergencia (ambulancias, etc.), cambiando el estado de los semáforos cuando es necesario. Utiliza ZeroMQ con el patrón REQ/REP para recibir y responder las consultas de estado y comandos de control provenientes de PC2. Además, mantiene una base de datos réplica de la base de datos en PC2 para asegurar la continuidad del sistema en caso de fallo de PC2.

Arquitectura Distribuida

- Comunicaciones: Los tres componentes (PC1, PC2 y PC3) se comunican de manera asíncrona utilizando ZeroMQ, que actúa como middleware para garantizar la eficiencia y escalabilidad del

sistema. Los patrones de mensajería utilizados son:

- PUB/SUB: Usado por PC1 para publicar eventos y por PC2 para suscribirse a estos eventos.
- REQ/REP: Usado entre PC2 y PC3 para enviar solicitudes de estado y recibir respuestas.
- Resiliencia y Tolerancia a Fallos:
 - Redundancia: Se incluye un esquema de replicación de base de datos entre PC2 y PC3, donde PC2 mantiene una copia de seguridad de los datos y puede asumir el rol de PC3 en caso de fallo.
 - Conmutación automática: Si PC3 falla, el sistema detectará la falla mediante un mecanismo de heartbeat y se activará la réplica en PC2 para continuar funcionando sin interrupciones.
- Escalabilidad: La arquitectura está diseñada para permitir la adición de más sensores y semáforos sin alterar la estructura básica del sistema. Los nuevos componentes pueden ser fácilmente integrados, lo que asegura la capacidad de manejar ciudades más grandes con más intersecciones.

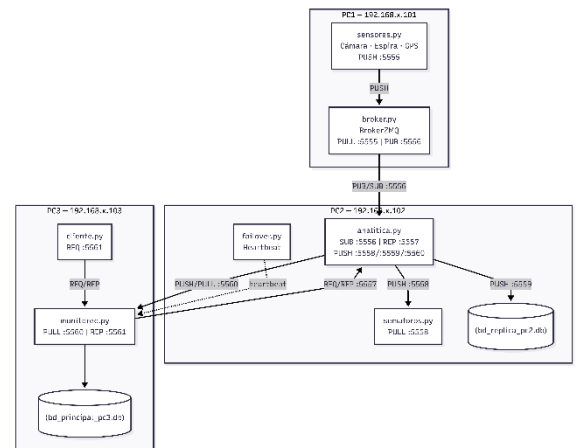


Fig. 1. Diagrama de Despliegue

B. Modelo de Interacción

El modelo de interacción describe cómo los procesos se comunican y coordinan entre sí. En este sistema existen dos tipos principales de interacción:

Interacción asíncrona (predominante):

- Los sensores generan y publican eventos de forma periódica e independiente, sin esperar confirmación. El broker recibe y reenvía sin bloquear a los sensores.
- El servicio de analítica envía comandos al controlador de semáforos de manera asíncrona (PUSH/PULL), lo que garantiza que el procesamiento de nuevos eventos no se vea bloqueado por la ejecución de acciones físicas.
- Las actualizaciones a la base de datos (principal y réplica) se realizan de forma asíncrona para maximizar el throughput.

Interacción síncrona (consultas de usuario):

- El Servicio de Monitoreo en PC3 utiliza el patrón REQ/REP para enviar consultas o indicaciones directas al Servicio de Analítica en PC2. La respuesta es esperada de forma bloqueante, lo cual es aceptable dado que estas operaciones son iniciadas por humanos y tienen baja frecuencia.

Interacción entre PC1 y PC2 (Pub/Sub para el flujo de datos): PC1 genera eventos de tráfico en tiempo real a través de sensores como cámaras, espiras inductivas y GPS. Estos eventos son publicados por PC1 a PC2 mediante ZeroMQ PUB/SUB. PC2 se suscribe a estos eventos para recibir los datos de tráfico, lo que le permite tomar decisiones sobre el control de los semáforos y gestionar el tráfico de forma eficiente.

Interacción entre PC2 y PC3 (Request/Reply para consultas de estado): PC2 consulta el estado del sistema y gestiona emergencias enviando solicitudes a PC3 mediante ZeroMQ REQ/REP. PC3 responde con información sobre los semáforos y las situaciones de emergencia. En caso de fallo de PC3, PC2 puede usar una base de datos réplica para garantizar la continuidad del servicio.

Control de semáforos en PC2: PC2 controla los semáforos en función de los eventos de tráfico. Si hay congestión, ajusta los semáforos para aliviarla. Si un vehículo de emergencia se aproxima, cambia el semáforo para permitir su paso, garantizando un flujo de tráfico eficiente y seguro en tiempo real.

Operaciones

• Detección de Congestión

Cuando los sensores de tráfico en PC1 detectan un aumento en el volumen vehicular o una disminución en la velocidad promedio, PC2 procesa estos datos para identificar congestionamiento. Si la congestión supera un umbral preestablecido, PC2 ajusta los semáforos para aliviar el tráfico, por ejemplo, extendiendo la fase verde en las intersecciones más congestionadas.

• Operación de Emergencia

Cuando PC1 detecta la aproximación de un vehículo de emergencia (como una ambulancia), PC2 cambia automáticamente el semáforo de la vía correspondiente a verde, permitiendo el paso del vehículo sin obstáculos. Esta operación asegura que los vehículos de emergencia puedan moverse rápidamente a través de la ciudad, priorizando su paso sobre el tráfico normal.

• Monitoreo de Tráfico y Semáforos

PC3 realiza un monitoreo constante del flujo de tráfico y el estado de los semáforos, recibiendo datos en tiempo real de PC2. Si se detectan problemas, como semáforos defectuosos o congestión inusual, PC3 alerta a PC2 para que se tomen medidas correctivas. Esta operación asegura que el sistema de tráfico esté siempre optimizado y que cualquier anomalía sea detectada rápidamente.

• Fallo de PC3

En caso de que PC3 falle, el sistema activa un failover hacia la Base de Datos Réplica en PC2, garantizando que el servicio siga funcionando sin

interrupciones. PC2 toma el control de las funciones de monitoreo y consulta de estado, asegurando que el sistema continúe gestionando el tráfico sin pérdida de datos o capacidad de respuesta.

C. Modelo de Fallos

1. Fallo en PC1 (Publicador de eventos de tráfico)

PC1, el nodo encargado de capturar los datos del tráfico, puede experimentar fallos debido a problemas en los sensores (como cámaras, espiras inductivas o sensores GPS), en la red o en el Broker ZeroMQ. Si PC1 deja de publicar eventos, PC2 no recibirá datos y, por lo tanto, no podrá tomar decisiones sobre el control de los semáforos.

- Manejo del fallo:
 - PC2 puede detectar que no recibe datos de PC1 y activar una alerta de fallo.
 - Si PC1 falla de manera prolongada, el sistema puede utilizar datos históricos almacenados en la base de datos de PC2 para continuar con la gestión básica del tráfico hasta que PC1 se recupere.
 - Resiliencia: Si PC1 no está disponible temporalmente, el sistema puede funcionar con datos previos, aunque se perderán eventos en tiempo real.

2. Fallo en PC2 (Procesamiento de eventos y control de semáforos)

PC2 es crucial para el procesamiento de los eventos recibidos y el control de los semáforos. Si PC2 falla, el sistema no podrá ajustar el estado de los semáforos ni realizar análisis de tráfico en tiempo real. Esto podría resultar en un control deficiente del tráfico y una falta de respuesta ante emergencias.

- Manejo del fallo:
 - Failover automático: Si PC2 experimenta un fallo, el sistema puede conmutar a una

Base de Datos Réplica en PC3, lo que permite a PC3 tomar control temporal de las operaciones de semáforos.

- Monitoreo: Si PC2 no responde, PC3 puede asumir algunas de las responsabilidades del monitoreo y la consulta de estado.
- Recuperación: Cuando PC2 se recupera, PC3 sincroniza los datos y vuelve a transferir el control a PC2.

3. Fallo en PC3 (Monitoreo y gestión de emergencias)

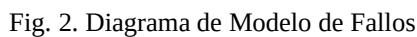
PC3 es el encargado del monitoreo general y la gestión de emergencias (como la priorización de vehículos de emergencia). Si PC3 experimenta un fallo, PC2 perderá la capacidad de consultar el estado del sistema y gestionar las emergencias.

- Manejo del fallo:
 - Failover y redundancia: PC2 tiene acceso a una Base de Datos Réplica en PC3. Si PC3 falla, PC2 puede seguir operando con la réplica de la base de datos hasta que PC3 se recupere.
 - Manejo manual de emergencias: En caso de que PC3 falle, PC2 puede manejar emergencias de forma manual, aunque con capacidad limitada.
 - Recuperación: Cuando PC3 se recupere, PC2 puede reestablecer la conexión y volver a realizar las consultas automáticas para el monitoreo y la gestión de emergencias.

4. Fallo en la Base de Datos (BD Principal y BD Réplica)

La Base de Datos Principal en PC3 almacena información crítica sobre el estado del sistema,

- Manejo del fallo:
 - Base de Datos Réplica: PC2 mantiene una Base de Datos Réplica en caso de que la base de datos principal falle. PC2 puede redirigir las consultas a la réplica mientras se resuelve el problema.
 - Sincronización de datos: Cuando la base de datos principal se recupera, se realiza una sincronización con la base de datos réplica para asegurar que toda la información esté actualizada.



- **Autenticación de Mensajes:** Los mensajes enviados entre los diferentes componentes del sistema incluyen un identificador único de origen, como `sensor_id` y `tipo_sensor`. Este identificador permite validar la procedencia de los datos y asegura que los eventos de tráfico recibidos en PC2 y PC3 provienen de sensores legítimos. La autenticidad de los datos es verificada antes de ser procesados o almacenados.
- **Integridad de Datos:** El sistema utiliza ZeroMQ, que garantiza la entrega completa de los mensajes entre los nodos, es decir, los mensajes se entregan en su totalidad o no se

- **Control de Acceso:** El Servicio de Monitoreo en PC3 es el único punto de acceso externo para realizar operaciones de control manual, como el cambio de semáforos en situaciones de emergencia o en ajustes específicos del sistema. Los demás servicios, como PC1 y PC2, solo aceptan conexiones de componentes internos, lo que garantiza que solo las operaciones internas autorizadas puedan modificar el estado del sistema. Este control asegura que no haya manipulaciones externas no deseadas.
- **Disponibilidad:** PC2 mantiene una Base de Datos Réplica que garantiza la disponibilidad continua del servicio de datos, incluso si PC3 experimenta un fallo. La réplica permite que PC2 continúe operando y procesando los eventos de tráfico sin interrupciones, asegurando que el sistema de control de semáforos y la toma de decisiones no se vean afectados por un fallo en PC3. Además, la sincronización de datos entre PC2 y PC3 asegura que la información esté siempre actualizada.



IV. Diseño del Sistema

A. Diagrama de Componentes

El sistema se despliega en tres nodos físicos desacoplados mediante ZeroMQ:

- PC1 (Ingesta y Distribución): Ejecuta sensores.py para simular la generación de eventos (Cámara, Espira, GPS) en 25 intersecciones. Los datos se envían a broker.py, que centraliza y redistribuye el tráfico de mensajes mediante patrones PUSH/PULL y PUB/SUB, permitiendo evaluar el rendimiento en arquitecturas mono y multihilo.
- PC2 (Procesamiento y Control): Aloja el servicio analitico.py, que procesa eventos bajo reglas lógicas para emitir comandos a semaforos.py. Gestiona la persistencia asíncrona hacia las bases de datos y cuenta con un módulo de failover.py que, mediante heartbeats, conmuta automáticamente a una réplica local si detecta la caída del tercer nodo.
- PC3 (Persistencia y Gestión): Contiene la base de datos principal (receptor PULL) y el servicio de monitoreo. A través de cliente.py, permite la gestión interactiva (REQ/REP) para consultas históricas, cambios manuales y priorización de emergencias. El esquema de datos se mantiene unificado mediante el módulo compartido database.py.

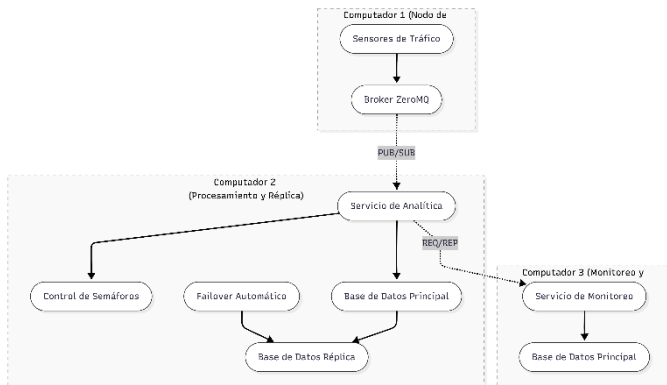


Fig. 4. Diagrama de Componentes

B. Diagrama de Clases

El diagrama muestra las clases reales del proyecto organizadas en cuatro paquetes según su nodo de ejecución.

Todas las clases que representan procesos concurrentes heredan de threading.Thread de Python, lo que les permite correr en paralelo dentro del mismo proceso sin bloquearse entre sí.

PC1 tiene HiloSensor, que crea un hilo por cada intersección de la cuadrícula generando los tres tipos de evento, y WorkerBroker, que forma el pool de workers del broker multihilo para redistribuir mensajes bajo alta carga.

PC2 tiene tres hilos en la analítica: HiloSuscriptor recibe eventos del broker y aplica las reglas de tráfico, HiloIndicacionesREP atiende indicaciones directas de PC3 mediante REQ/REP, y HiloReplicaBD persiste los datos en la BD réplica local. Además EstadoPC3 mantiene el estado de disponibilidad de PC3 y HiloHeartbeat lo monitorea enviando pings cada 3 segundos para activar el failover automático.

PC3 tiene HiloPersistencia que recibe todos los datos de PC2 y los guarda en la BD principal, y HiloMonitoreoREP que atiende las consultas del usuario y reenvía indicaciones a PC2.

shared contiene Database con todas las operaciones SQLite usadas por PC2 y PC3, y Config con todas las constantes del sistema importadas por todos los módulos al arrancar.

Las cinco tablas SQLite representan el esquema de datos compartido entre la BD principal en PC3 y la BD réplica en PC2.

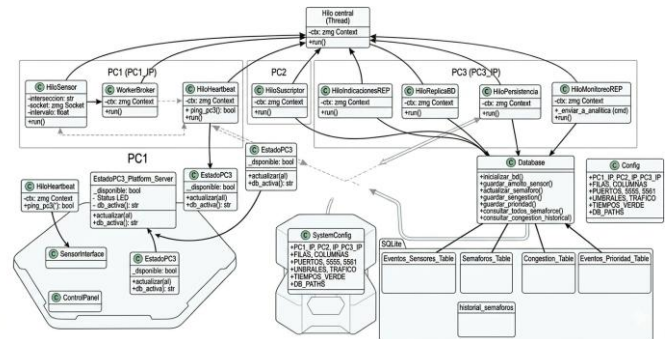


Fig.5. Diagrama de Clases

C. Diagrama de Secuencia

El diagrama de secuencia muestra los tres flujos principales del sistema:

Flujo asíncrono (tráfico normal): el sensor envía el evento al Broker vía PUSH, este lo reenvía a Analítica vía PUB. Analítica persiste el evento en ambas bases de datos de forma asíncrona y envía el comando de

semáforo Verde 15s. El semáforo cambia automáticamente a rojo al expirar el temporizador, sin intervención adicional.

Detección de congestión: cuando Analítica recibe un evento que supera los umbrales definidos ($Q \geq 5$ o $V_p \leq 35$), registra la condición de congestión en BD Réplica y BD Principal, y emite un comando de Verde extendido a 30 segundos para liberar el flujo vehicular acumulado en la intersección.

Priorización de emergencia (síncrono (REQ/REP)): es el único flujo síncrono del sistema. El operador en PC3 envía una solicitud PRIORIDAD_EMERGENCIA directamente a Analítica, que aplica la ola verde de 45 segundos y responde con un ACK confirmando la acción. Este patrón REQ/REP garantiza que el operador recibe confirmación explícita antes de continuar.

El diseño es correcto para la sustentación: separa claramente los flujos asíncronos del síncrono, usa los puertos definidos en la configuración y evidencia los tres estados de semaforización exigidos por el enunciado.

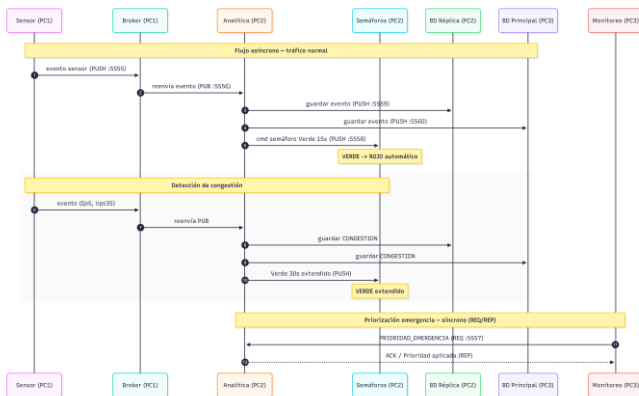


Fig. 6. Diagrama de Secuencia Interacciones Asíncronas y Síncronas

D. Reglas de Tráfico

El sistema define tres estados de circulación implementados en analítica.py mediante la función `_determinar_condicion()`. Los umbrales están configurados en `shared/config.py`.

- Estado 1: Tráfico Normal

Se activa cuando las tres variables cumplen simultáneamente: $Q < 5$ AND $V_p > 35$ AND $D < 20$

Donde Q es la longitud de cola en vehículos (sensor cámara), V_p es la velocidad promedio en km/h (cámara o GPS) y D es la densidad en veh/km (GPS). Al detectarse este estado la analítica emite un verde de 15 segundos e imprime: [ANALITICA] INT_B2 | NORMAL ($Q=3$, $V_p=42.5$, $D=12.3$) → Verde 15s

- Estado 2: Congestión

Se activa cuando al menos una variable supera su umbral: $Q \geq 5$ OR $V_p \leq 35$ OR $D \geq 20$

La analítica extiende la fase verde a 30 segundos para aliviar la intersección afectada y registra el evento en la tabla congestion de ambas BDs. Imprime: [ANALITICA] INT_C1 | CONGESTION ($Q=12$, $V_p=18.4$, $D=34.7$) → Verde extendido 30s

- Estado 3: Priorización de vía (ola verde)

Se activa únicamente por indicación directa del usuario desde cliente.py. No depende de los valores de los sensores. La analítica activa una ola verde de 60 segundos en la intersección indicada y registra el evento en la tabla eventos_prioridad. Imprime: [ANALITICA-REP] PRIORIDAD ambulancia en INT_B2 → ola verde 60s

V. Plan de Métricas para el Proyecto

El plan de métricas define con precisión qué variables se medirán, bajo qué condiciones se recolectarán los datos y qué resultados se esperan obtener en cada escenario de prueba. Su propósito es establecer un marco de referencia claro antes de ejecutar cualquier experimento, garantizando que las mediciones sean reproducibles y comparables entre escenarios.

El enunciado establece dos variables dependientes que deben medirse bajo dos escenarios de carga distintos, comparando el diseño original con broker de un solo hilo frente al diseño

cliente reciba la respuesta de confirmación. Finalmente se ejecutan todas las opciones del menú de cliente.py verificando que cada consulta retorne datos coherentes: estado de semáforos, historial de congestión filtrado por fecha e intersección, historial de cambios de semáforo y eventos de priorización registrados.

- Pruebas de integración.

Con los tres nodos corriendo en máquinas virtuales separadas y las IPs configuradas correctamente, se verifica que la analítica de PC2 reciba eventos provenientes de los sensores de PC1 a través del broker. Se comprueba que los datos enviados por la analítica de PC2 lleguen a la BD principal de

PC3 consultando directamente la tabla. Se realiza una operación de PRIORIDAD_EMERGENCIA desde el cliente en PC3 y se verifica que la indicación llegue correctamente a la analítica de PC2, que esta emita el comando al controlador y que la respuesta de confirmación regrese al cliente pasando por toda la cadena de componentes distribuidos.

- Pruebas de desempeño.

Con la funcionalidad verificada se procede a las pruebas de desempeño siguiendo los dos escenarios de la Tabla 1. En cada escenario se arranca el sistema con la configuración correspondiente, se espera el período de calentamiento de 60 segundos y se ejecuta `python3 metricas.py` para recolectar M1 y M2 de forma automática. El procedimiento completo se repite tres veces por escenario para obtener mediciones independientes. Los resultados se anotan en la tabla de registro

modificado con broker multihilo. El sistema opera sobre una cuadrícula de 3×3, lo que produce 9 intersecciones activas con tres tipos de sensor cada una (cámara, espira inductiva y GPS), generando 27 eventos por ciclo de medición.

Los dos factores que se varían entre escenarios son el intervalo de generación de eventos, controlado mediante el argumento de línea de comandos `sensores.py`, y el modo de operación del broker, controlado mediante el argumento de `broker.py`.

- Métrica M1 — Cantidad de eventos almacenados en BD en 2 minutos. Esta métrica cuantifica el rendimiento efectivo del sistema, es decir, cuántos eventos generados por los sensores logran llegar completos a la base de datos en un intervalo fijo de 120 segundos. Refleja directamente la capacidad del broker y de la analítica para procesar y persistir datos bajo distintas cargas. Los valores teóricos esperados son los siguientes. En el Escenario A se producen 27 eventos cada 10 segundos, lo que equivale a 324 eventos en 2 minutos. En el Escenario B se producen 27 eventos cada 5 segundos, lo que equivale a 648 eventos en 2 minutos. Si el broker multihilo funciona correctamente, el Escenario B debería acercarse más a su valor teórico que el Escenario A con broker simple, porque el grupo de 4 workers reduce la cola de mensajes bajo alta carga y evita que se descarten eventos por saturación del socket.
- Métrica M2 — Tiempo desde que el usuario solicita una acción hasta que el semáforo cambia. Esta métrica cuantifica la latencia extremo a extremo del

definida en el plan de métricas y se construyen los gráficos comparativos requeridos para el informe de la segunda entrega.

- Pruebas de tolerancia a fallos.

Con el sistema corriendo bajo carga normal del Escenario A, se detiene el proceso `monitoreo.py` en PC3 y se observa en la terminal de `failover.py` que la detección ocurre tras dos heartbeats fallidos consecutivos en un tiempo máximo de 9 segundos. Se verifica que el procesamiento de eventos continúe sin interrupciones usando la BD réplica en PC2 y que los semáforos sigan cambiando de estado durante el período de falla. A continuación se reinicia `monitoreo.py` en PC3 y se confirma que el sistema detecte la recuperación en el siguiente ciclo de heartbeat de 3 segundos y vuelva a usar la BD principal automáticamente.

camino síncrono del sistema: desde que el usuario envía el comando PRIORIDAD_EMERGENCIA desde `metricas.py` hasta que el cambio de estado VERDE queda reflejado en la base de datos.

Se reportan promedio, mínimo y máximo sobre 5 repeticiones por escenario. Los valores esperados en condiciones de red local entre máquinas virtuales oscilan entre 50 y 500 milisegundos. Si el tiempo supera 1 segundo indica congestión en PC2 o latencia de red elevada.

VI. Protocolo de Pruebas

El protocolo establece la secuencia de pruebas que se ejecutará sobre el sistema, los criterios de aceptación para cada tipo de prueba y las condiciones del entorno de ejecución. Su objetivo es validar correctitud funcional, capacidad de procesamiento, resiliencia ante fallos y consistencia de datos de extremo a extremo.

Para replicar un entorno realista se trabaja con tres máquinas virtuales. En PC1 residen los sensores simulados y el broker ZeroMQ. En PC2 operan la analítica, el controlador de semáforos, el módulo de failover y la BD réplica. En PC3 se ejecutan el servicio de monitoreo, la BD principal y la consola cliente.

- Pruebas funcionales.

Primero se valida que cada tipo de sensor genere eventos con la estructura correcta y los campos requeridos, que el `sensor_id` corresponda al tipo y la intersección, y que los valores numéricos estén dentro de los rangos definidos. Se verifica que las 9 intersecciones de la cuadrícula 3×3 aparezcan en la terminal de `analitica.py` durante los primeros 30 segundos de operación. A continuación se

confirma la detección de congestión inyectando eventos con $Q \geq 5$, $V_p \leq 35$ y $D \geq 20$ y verificando que la analítica clasifique la intersección como CONGESTION y emita el comando de verde extendido de 30 segundos. Luego se inyectan valores dentro del rango normal y se confirma la clasificación NORMAL con verde de 15 segundos. Para la priorización de emergencia se envía el comando PRIORIDAD_EMERGENCIA desde cliente.py y se verifica que la ola verde de 60 segundos se active en la intersección indicada, que el evento quede registrado en la tabla eventos_prioridad de ambas bases de datos y que el

VII. Como se obtendrán las Métricas de Desempeño

Las métricas se obtienen combinando el script metricas.py incluido en el proyecto con observación directa de las terminales de cada proceso y consultas SQL directas a las bases de datos SQLite. A continuación se describe el procedimiento técnico para cada métrica.

- **Obtención de M1**

Eventos almacenados en 2 minutos. La función contar_eventos_bd(ruta_db, segundos=120) en metricas.py ejecuta la siguiente consulta directamente sobre la base de datos indicada:

```
SELECT COUNT(*) FROM eventos_sensores
WHERE recibido_en >=
'<timestamp_hace_120_segundos>'
```

El campo recibido_en corresponde al instante de llegada real al módulo de persistencia, calculado en shared/database.py mediante _ahora(), y no al timestamp generado por el sensor. Esto garantiza que se mide exactamente lo que fue persistido en el intervalo de 120 segundos, independientemente de la latencia de red o de los desfases de reloj entre nodos. La función se ejecuta tanto sobre bdprincipal como sobre bd replica para obtener los dos conteos por separado. Antes de cada escenario se eliminan ambas bases de datos con rm -f bd_principal_pc3.db bd_replica_pc2.db para garantizar que el conteo refleje únicamente los eventos del experimento en curso.

- **Obtención de M2**

Tiempo de respuesta del semáforo. La función medir_tiempo_cambio_semaforo("INT_C3", ruta_db, timeout_seg=10) en metricas.py registra el timestamp del sistema con time.time() justo antes de enviar el comando PRIORIDAD_EMERGENCIA al puerto 5561 de PC3,

espera el ACK del servicio de monitoreo y luego realiza polling cada 100 milisegundos sobre la tabla semaforos consultando:

```
SELECT estado FROM semaforos WHERE interseccion =
'INT_C3'
```

Cuando el campo estado cambia a VERDE, calcula la diferencia con el timestamp inicial y retorna el tiempo transcurrido en segundos. Si no se detecta el cambio dentro de los 10 segundos de timeout retorna -1 y la repetición se marca como fallida. Se ejecutan 5 repeticiones con pausa de 3 segundos entre cada una para que el temporizador automático de semaforos.py devuelva el semáforo a ROJO antes de la siguiente medición. Al finalizar las 5 repeticiones se calculan el promedio, mínimo y máximo de los valores válidos.

VIII. Definición Inicial de Recursos

Todos los procesos del sistema obtienen su configuración inicial importando el módulo shared/config.py al momento de arrancar. Este archivo centraliza todas las constantes del sistema, lo que significa que cualquier cambio en la configuración se propaga automáticamente a todos los nodos sin modificar ningún otro archivo.

La cuadrícula de la ciudad se define mediante dos listas:

```
pythonFILAS = ["A", "B", "C"]
COLUMNAS = [1, 2, 3]
```

La función lista_intersecciones() en sensores.py combina estas listas automáticamente generando las 9 intersecciones: INT_A1, INT_A2, INT_A3, INT_B1, INT_B2, INT_B3, INT_C1, INT_C2, INT_C3. Por cada intersección se crea un hilo HiloSensor que simula los tres tipos de sensor, por lo que el número total de sensores activos es siempre 9 intersecciones $\times$ 3 tipos = 27 sensores. El número de semáforos es igual al número de intersecciones: 9 en total, uno por intersección.

El intervalo de generación de eventos se pasa como argumento al arrancar sensores.py:

```
bashpython3 sensores.py 10 # Escenario A
python3 sensores.py 5 # Escenario B
```

Si no se pasa argumento usa el valor por defecto definido en config.py (INTERVALO_SENSOR_SEG = 5).

IX. Reglas, Consultas e Indicaciones Directas

- Reglas de tráfico

La analítica evalúa cada intersección combinando la última lectura de los tres sensores y aplica las siguientes reglas definidas en config.py:

Estado y Condición

Verde NORMAL -> $Q < 5$ AND $V_p > 35$ AND $D < 20$ -> 15s

CONGESTION -> $Q \geq 5$ OR $V_p \leq 35$ OR $D \geq 20$ -> 30s

PRIORIDAD -> Emergencia declarada -> 60s

Tipos de consulta del usuario

Desde cliente.py el usuario puede realizar las siguientes consultas al servicio de monitoreo de PC3:

```
{"accion": "GET_SEMAFOROS"}
```

```
{"accion": "GET_SEMAFORO", "interseccion": "INT_B2"}
```

```
{"accion": "GET_CONGESTION", "desde": "2026-04-09T07:00:00Z", "hasta": "2026-04-09T09:00:00Z"}
```

```
{"accion": "GET_HISTORIAL", "interseccion": "INT_B2", "limite": 50}
```

```
{"accion": "GET_PRIORIDADES"}
```

```
{"accion": "HEARTBEAT"}
```

- Indicaciones directas del Monitoreo a la Analítica

Cuando el usuario envía una indicación desde cliente.py, el servicio de monitoreo en PC3 la recibe y la reenvía directamente a la analítica en PC2 mediante REQ/REP en el puerto 5557. Ejemplos:

Prioridad de emergencia — activa ola verde de 60 segundos:

```
{
  "accion": "PRIORIDAD_EMERGENCIA",
  "interseccion": "INT_B2",
  "tipo_vehiculo": "ambulancia"
}
```

Cambio manual de semáforo — fuerza un estado independiente de los sensores:

```
{
  "accion": "CAMBIO_SEMAFORO",
```

```
"interseccion": "INT_A3",
```

```
"estado": "ROJO"
```

```
}
```

Consulta de estado — solicita el estado actual de todos los semáforos directamente a la analítica:

```
{"accion": "CONSULTA_ESTADO"}
```

X. Conclusiones

El sistema distribuido de gestión inteligente de tráfico urbano logra integrar efectivamente los principios de comunicación asíncrona, tolerancia a fallos y persistencia distribuida en una arquitectura de tres nodos usando ZeroMQ como middleware de mensajería. La separación de responsabilidades entre PC1 (generación de eventos), PC2 (procesamiento y control) y PC3 (persistencia y monitoreo) produce un sistema desacoplado que puede escalar horizontalmente sin afectar a los demás componentes.

La combinación de patrones PUB/SUB para el flujo continuo de eventos de sensores y REQ/REP para las indicaciones directas del usuario demostró ser adecuada para las características del problema, garantizando alto throughput en el flujo asíncrono y respuesta predecible en el flujo síncrono, especialmente en situaciones de emergencia donde la latencia debe ser mínima y acotada.

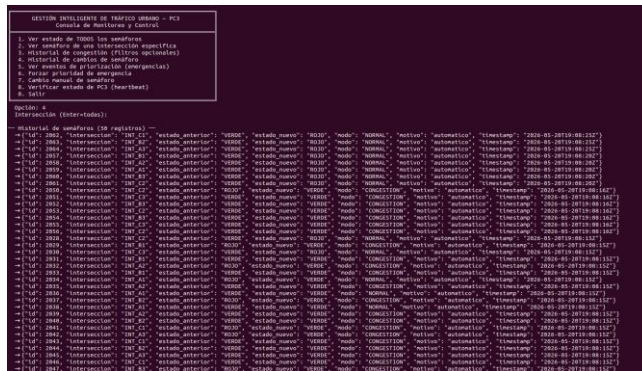
El mecanismo de failover implementado mediante heartbeat cada 3 segundos y conmutación automática a la BD réplica en un máximo de 9 segundos garantiza la continuidad del servicio ante la caída de PC3 de forma completamente transparente para el cliente, cumpliendo con el requisito principal del enunciado respecto a la tolerancia a fallos.

El diseño del broker en modo simple y multihilo responde directamente al requisito de comparación de desempeño de la Tabla 1. Se espera que el modo multihilo demuestre mayor throughput en el Escenario B de alta carga, evidenciando cómo la paralelización interna del broker reduce el cuello de botella bajo alta frecuencia de generación de eventos y justificando su uso en entornos de mayor escala.

En conjunto, el proyecto demuestra la viabilidad de implementar una arquitectura distribuida orientada a eventos capaz de gestionar condiciones de tráfico urbano en tiempo real, con mecanismos de resiliencia, persistencia dual y una interfaz de consulta que permite auditar completamente el comportamiento del sistema en cualquier momento.

[illegible]

- Historial de semáforos



- Eventos priorización



• Pruebas Desempeño

Prueba 1: 1 sensor de cada tipo generado datos en intervalos de 10s con bróker simple.

```
estudiante@MIG272:~/trafico2$ python3 metricas.py

MÉTRICAS DE RENDIMIENTO – Tabla 1 del Proyecto ISD
Pontificia Universidad Javeriana

Matriz de pruebas (4 combinaciones):
Carga A: 1 sensor/tipo, intervalo=10s
Carga B: 2 sensores/tipo, intervalo=5s
Broker: simple | multithlo

Carga aplicada por los sensores (A / B): A
Broker que está corriendo (simple / multithlo): simple
¿Qué BD usar? (principal / replica) [principal]: principal
Número de repeticiones para M2 [5]: 5

CONFIRMAR ANTES DE CONTINUAR:
Carga: A (1 sensor de cada tipo, intervalo=10s )
Broker: simple
BD: bd_principal_pc3.db
Reps. M2: 5

△Verifique en la Terminal 4 que el banner del broker coincide:
- simple →[BROKER-SIMPLE]
- multithlo →[BROKER-MULTITHILO] workers=N

¿Continuar? (s/n): s

[20:30:13] Iniciando mediciones. BD: /home/estudiante/trafico2/pc3/bd_principal_pc3.db

=====
ESCENARIO: Carga A + Broker simple
1 sensor de cada tipo, intervalo=10s, broker en modo simple
BD: /home/estudiante/trafico2/pc3/bd_principal_pc3.db
=====

[20:30:13] Esperando 120s para medir throughput...
[20:30:13] (Asegúrese de que los sensores estén generando datos)
[20:32:14] ✅ M1 – Eventos en 2 min: 324 | Tasa: 2.7 evt/s

[20:32:14] Midiendo latencia para INT_C3 (5 repeticiones)...
Repetición 1/5: 0.017s
Repetición 2/5: 0.026s
Repetición 3/5: 0.018s
Repetición 4/5: 0.070s
Repetición 5/5: 0.019s
[20:32:29] ✅ M2 – Latencia: avg=0.03s | min=0.0172s | max=0.0699s (5/5 exitosas)
```

Prueba 2: 1 sensor de cada tipo generado datos en intervalos de 10s con bróker multihilo

```
estudiante@MIG272:~/trafico2$ python3 metricas.py

MÉTRICAS DE RENDIMIENTO – Tabla 1 del Proyecto ISD
Pontificia Universidad Javeriana

Matriz de pruebas (4 combinaciones):
Carga A: 1 sensor/tipo, intervalo=10s
Carga B: 2 sensores/tipo, intervalo=5s
Broker: simple | multithlo

Carga aplicada por los sensores (A / B): A
Broker que está corriendo (simple / multithlo): multithlo
¿Qué BD usar? (principal / replica) [principal]: principal
Número de repeticiones para M2 [5]: 5

CONFIRMAR ANTES DE CONTINUAR:
Carga: A (1 sensor de cada tipo, intervalo=10s )
Broker: multithlo
BD: bd_principal_pc3.db
Reps. M2: 5

△Verifique en la Terminal 4 que el banner del broker coincide:
- simple →[BROKER-SIMPLE]
- multithlo →[BROKER-MULTITHILO] workers=N

¿Continuar? (s/n): s

[20:39:36] Iniciando mediciones. BD: /home/estudiante/trafico2/pc3/bd_principal_pc3.db

=====
ESCENARIO: Carga A + Broker multithlo
1 sensor de cada tipo, intervalo=10s, broker en modo multithlo
BD: /home/estudiante/trafico2/pc3/bd_principal_pc3.db
=====

[20:39:36] Esperando 120s para medir throughput...
[20:39:36] (Asegúrese de que los sensores estén generando datos)
[20:41:30] ✅ M1 – Eventos en 2 min: 305 | Tasa: 2.542 evt/s

[20:41:36] Midiendo latencia para INT_C3 (5 repeticiones)...
Repetición 1/5: 0.031s
Repetición 2/5: 0.071s
Repetición 3/5: 0.014s
Repetición 4/5: 0.016s
Repetición 5/5: 0.016s
[20:41:51] ✅ M2 – Latencia: avg=0.0297s | min=0.014s | max=0.0713s (5/5 exitosas)
```

Prueba 3: 2 sensor de cada tipo generado datos en intervalos de 5s con bróker simple

```
estudiante@MIG272:~/trafico2$ python3 metricas.py

MÉTRICAS DE RENDIMIENTO – Tabla 1 del Proyecto ISD
Pontificia Universidad Javeriana

Matriz de pruebas (4 combinaciones):
Carga A: 1 sensor/tipo, intervalo=10s
Carga B: 2 sensores/tipo, intervalo=5s
Broker: simple | multithlo

Carga aplicada por los sensores (A / B): B
Broker que está corriendo (simple / multithlo): simple
¿Qué BD usar? (principal / replica) [principal]: principal
Número de repeticiones para M2 [5]: 5

CONFIRMAR ANTES DE CONTINUAR:
Carga: B (2 sensores de cada tipo, intervalo=5s )
Broker: simple
BD: bd_principal_pc3.db
Reps. M2: 5

△Verifique en la Terminal 4 que el banner del broker coincide:
- simple →[BROKER-SIMPLE]
- multithlo →[BROKER-MULTITHILO] workers=N

¿Continuar? (s/n): s

[20:47:25] Iniciando mediciones. BD: /home/estudiante/trafico2/pc3/bd_principal_pc3.db

=====
ESCENARIO: Carga B + Broker simple
2 sensores de cada tipo, intervalo=5s, broker en modo simple
BD: /home/estudiante/trafico2/pc3/bd_principal_pc3.db
=====

[20:47:25] Esperando 120s para medir throughput...
[20:47:25] (Asegúrese de que los sensores estén generando datos)
[20:49:25] ✅ M1 – Eventos en 2 min: 1216 | Tasa: 10.133 evt/s

[20:49:25] Midiendo latencia para INT_C3 (5 repeticiones)...
Repetición 1/5: 0.071s
Repetición 2/5: 0.004s
Repetición 3/5: 0.025s
Repetición 4/5: 0.016s
Repetición 5/5: 0.070s
[20:49:41] ✅ M2 – Latencia: avg=0.0535s | min=0.0162s | max=0.0838s (5/5 exitosas)
```


Prueba 1: 2 sensor de cada tipo generado datos en intervalos de 5s con bróker multihilo

```
Resultados guardados en: resultados_simple_5s.json
estudiante@MIG272:~/tráfico$ python3 metricas.py

MÉTRICAS DE RENDIMIENTO - Tabla 1 del Proyecto ISD
Pontificia Universidad Javeriana

Matriz de pruebas (4 combinaciones):
Carga A: 1 sensor/tipo, intervalo=10s
Carga B: 2 sensores/tipo, intervalo=5s
Broker: simple | multihilo

Carga aplicada por los sensores (A / B): B
Broker que está corriendo (simple / multihilo): multihilo
¿Que BD usar? (principal / replica) [principal]: principal
Número de repeticiones para M2 [5]: 5

CONFIRMAR ANTES DE CONTINUAR:
Carga:      B (2 sensores de cada tipo, intervalo=5s)
Broker:     multihilo
BD:         bd_principal_pc3.db
Reps. M2:   5

△ Verifique en la Terminal 4 que el banner del broker coincide:
- simple → [BROKER-SIMPLE]
- multihilo → [BROKER-MULTIHIL0] workers=4

¿Continuar? (s/n): s

[20:55:09] Iniciando mediciones. BD: /home/estudiante/trafico2/pc3/bd_principal_pc3.db

=====
ESCENARIO: Carga B + Broker multihilo
2 sensores de cada tipo, intervalo=5s, broker en modo multihilo
BD: /home/estudiante/trafico2/pc3/bd_principal_pc3.db
=====

[20:55:09] Esperando 120s para medir throughput...
[20:55:09] (Asegúrese de que los sensores estén generando datos)
[20:57:10] M1 - Eventos en 2 min: 1212 | Tasa: 10.1 evt/s

[20:57:10] Midiendo latencia para INT_C3 (5 repeticiones)...
Repetición 1/5: 0.051s
Repetición 2/5: 0.031s
Repetición 3/5: 0.019s
Repetición 4/5: 0.071s
Repetición 5/5: 0.024s
[20:57:25] M2 - Latencia: avg=0.039s | min=0.0185s | max=0.0707s (5/5 exitosas)
```

XII. Referencias

[1] P. Hintjens, *ZeroMQ: Messaging for Many Applications*. Sebastopol, CA: O'Reilly Media, 2013.

[2] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. London, UK: Pearson, 2017.

[3] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA: Addison-Wesley, 2012.

[4] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media, 2017.

[5] ZeroMQ Community, "ZeroMQ Documentation — Socket Types and Patterns," *zeromq.org*. [Online]. Available:

<https://zeromq.org/socket-api/>. [Accessed: Apr. 9, 2026].

[6] ZeroMQ Community, "ZeroMQ Guide — Messaging Patterns," *zeromq.org*. [Online]. Available: <https://zguide.zeromq.org/>. [Accessed: Apr. 9, 2026].

[7] Python Software Foundation, "threading — Thread-based parallelism," *docs.python.org*. [Online]. Available: <https://docs.python.org/3/library/threading.html>. [Accessed: Apr. 9, 2026].

[8] Python Software Foundation, "sqlite3 — DB-API 2.0 interface for SQLite databases," *docs.python.org*. [Online]. Available: <https://docs.python.org/3/library/sqlite3.html>. [Accessed: Apr. 9, 2026].

[9] Python Software Foundation, "pyzmq — Python bindings for ZeroMQ," *pyzmq.readthedocs.io*. [Online]. Available: <https://pyzmq.readthedocs.io/en/latest/>. [Accessed: Apr. 9, 2026].

[10] D. R. Hipp, "SQLite: A Self-Contained, Serverless, Zero-Configuration, Transactional SQL Database Engine," *sqlite.org*. [Online]. Available: <https://www.sqlite.org/index.html>. [Accessed: Apr. 9, 2026].

[11] E. A. Brewer, "Towards robust distributed systems," in *Proc. 19th Annual ACM Symp. Principles of Distributed Computing (PODC)*, Portland, OR, USA, Jul. 2000, p. 7.

[12] M. Singhal and N. G. Shivaratri, *Advanced Concepts in Operating Systems*. New York, NY: McGraw-Hill, 1994.

[13] R. Guerraoui and L. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*, 2nd ed. Berlin, Germany: Springer, 2011.

[14] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[15] D. E. Comer, *Internetworking with TCP/IP, Vol. 1: Principles, Protocols, and Architecture*, 6th ed. Upper Saddle River, NJ: Pearson, 2014.

[16] Oracle Corporation, "VirtualBox — User Manual," *virtualbox.org*. [Online]. Available: <https://www.virtualbox.org/manual/>. [Accessed: Apr. 9, 2026].

[17] Canonical Ltd., "Ubuntu 22.04 LTS Server Documentation," *ubuntu.com*. [Online]. Available: <https://ubuntu.com/server/docs>. [Accessed: Apr. 9, 2026].

[18] M. Ben-Ari, *Principles of Concurrent and Distributed Programming*, 2nd ed. Harlow, UK: Addison-Wesley, 2006.

[19] N. A. Lynch, *Distributed Algorithms*. San Francisco, CA: Morgan Kaufmann, 1996.